Memoria Compartida (Shared Memory)

Malak Sanchez
Monitorias Sistemas Operativos

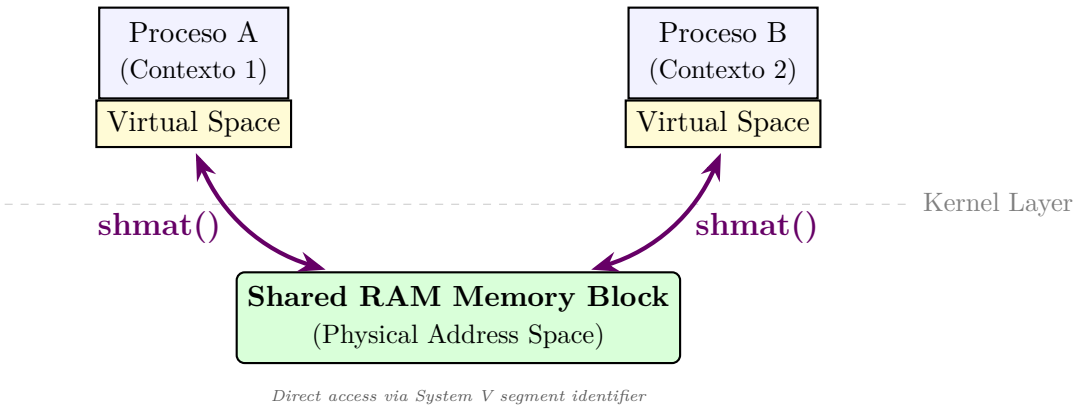
11 de octubre de 2026

1. Motivación

Cuando necesitamos transferir grandes volúmenes de datos entre procesos de manera ultra-rápida, los pipes son lentos porque implican copiar datos entre el espacio de usuario y el kernel. La **Memoria Compartida** permite que dos procesos mapeen la misma región de memoria física a su espacio de direcciones virtual.

2. Idea Fundamental

Normalmente, el kernel garantiza que un proceso no pueda acceder a la memoria de otro. La memoria compartida es una excepción controlada:



1. Se crea un segmento de memoria en el kernel.
2. Cada proceso lo "mapea"(adjunta) a su propio espacio virtual.
3. Los procesos leen y escriben como si fuera memoria local normal.

3. Sintaxis (Estilo System V)

```c
#include <sys/shm.h>
#include <sys/ipc.h>

// 1. Get identifier (Create segment)
int shmid = shmget(key, size, IPC_CREAT | 0666);

// 2. Attach to process address space
void *ptr = (void *)shmat(shmid, NULL, 0);

// 3. Detach when finished
shmdt(ptr);

// 4. Control/Removal
shmctl(shmid, IPC_RMID, NULL);
```

Descripción de las operaciones

- **shmget():** Crea u obtiene un segmento de memoria compartida identificado por una clave (*key*). Devuelve un identificador del segmento (*shmid*) que será utilizado en las operaciones posteriores.
- **shmat():** Adjunta el segmento de memoria compartida al espacio de direcciones del proceso y devuelve un puntero que permite acceder a dicha memoria como si fuera memoria normal.
- **shmdt():** Desvincula el segmento de memoria compartida del proceso una vez que este ha terminado de utilizarlo.
- **shmctl():** Permite realizar operaciones de control sobre el segmento; comúnmente se usa con `IPC_RMID` para marcar el segmento para su eliminación del sistema.

4. Ejemplo: Contador Compartido

Este ejemplo muestra la creación de un segmento para un entero compartido que es modificado por un proceso padre y su hijo simultáneamente.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main() {
    int shmid, *counter;
    int max_iter = 1000000;

    // Create shared memory segment (System V)
    shmid = shmget(IPC_PRIVATE, sizeof(int), 0666 | IPC_CREAT);
    counter = (int *)shmat(shmid, NULL, 0);
    *counter = 0;

    if(fork()){ // Parent process
        for(int i=0; i < max_iter; i++){
            *counter = *counter + 1;
        }

        wait(NULL);
        printf("Final value of counter: %d\n", *counter);

        shmdt(counter); // Detach from process address space
        shmctl(shmid, IPC_RMID, NULL); // Remove shared memory
           segment

    }else{// Child process
        for (int i = 0; i < max_iter; i++) *counter = *counter +
           1;

        shmdt(counter); // Detach from process address space
    }

    return EXIT_SUCCESS;
}
```

5. Precaución: El Problema de la Carrera

La memoria compartida **no tiene sincronización automática**. Si ambos procesos escriben al mismo tiempo (como en el ejemplo anterior), el resultado final podría no ser el esperado debido a condiciones de carrera. Para esto necesitaremos algún mecanismo de sincronización.

6. Ejercicios

1. Modifique el ejemplo para que cada hijo decremente el contador compartido. Ejecútelo varias veces y explique por qué el resultado puede diferir del esperado.

2. Compare la creación de un segmento con `IPC_PRIVATE` frente a una clave obtenida con `ftok()`; indique un caso de uso para cada alternativa.

3. Investigue el comando `ipcrm` para eliminar un segmento de memoria compartida e indique cómo identificar el segmento objetivo de forma segura.